SQL

If You Can't Answer These 20 SQL Questions, Don't Apply Yet

20 SQL questions from real data analyst interviews, easy to hard, each with a working answer. Try each one before you read the answer.

INSIDE THIS PDF · 9 PAGES

- The tables used
- Questions 1 to 4
- Questions 5 to 8
- Questions 9 to 12
- Questions 13 to 16
- Questions 17 and 18
- Questions 19 and 20

@thedataguy16

The tables used

All 20 answers use these tables. Answers are in PostgreSQL syntax.

```
employees(emp_id, name, dept, salary,
manager_id, hire_date)
orders(order_id, customer_id, order_date,
amount, status)
customers(customer_id, name, city, signup_date)
logins(user_id, login_date)
```

How to score

Count how many you solve without looking. The scoring guide is on the last page.

@thedataguy16

Questions 1 to 4

Basics. Every round starts here.

1. Employees earning above 50,000 in Sales

```
SELECT name, salary FROM employees
WHERE dept = 'Sales' AND salary > 50000;
```

2. Number of employees per department

```
SELECT dept, COUNT(*) AS headcount
FROM employees GROUP BY dept;
```

3. Departments with average salary above 60,000

HAVING filters after grouping.

```
SELECT dept, AVG(salary) FROM employees
GROUP BY dept HAVING AVG(salary) > 60000;
```

4. Customers with no city recorded

= NULL never matches. Use IS NULL.

```
SELECT * FROM customers WHERE city IS NULL;
```

@thedataguy16

Questions 5 to 8

Joins.

5. Each order with the customer name

```
SELECT o.order_id, c.name, o.amount
FROM orders o
JOIN customers c
  ON c.customer_id = o.customer_id;
```

6. Customers who never ordered

```
SELECT c.* FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

7. Employees with their manager's name

Self join. LEFT keeps the CEO who has no manager.

```
SELECT e.name, m.name AS manager
FROM employees e
LEFT JOIN employees m
  ON m.emp_id = e.manager_id;
```

8. Employees earning more than their manager

```
SELECT e.name FROM employees e
JOIN employees m
  ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

@thedataguy16

Questions 9 to 12

Aggregation with logic.

9. Revenue per city, highest first

```
SELECT c.city, SUM(o.amount) AS revenue
FROM orders o
JOIN customers c USING (customer id)
GROUP BY c.city
ORDER BY revenue DESC;
```

10. Delivered vs cancelled orders per month

```
SELECT DATE_TRUNC('month', order_date) AS m,
  COUNT(*) FILTER
    (WHERE status='Delivered') AS delivered,
  COUNT(*) FILTER
    (WHERE status='Cancelled') AS cancelled
FROM orders
GROUP BY 1 ORDER BY 1;
```

11. Each city's share of total revenue %

```
SELECT c.city,
  ROUND(100.0 * SUM(o.amount)
    / SUM(SUM(o.amount)) OVER (), 1) AS pct
FROM orders o
JOIN customers c USING (customer_id)
GROUP BY c.city;
```

12. Duplicate customer names

```
SELECT name, COUNT(*) FROM customers
GROUP BY name HAVING COUNT(*) > 1;
```

@thedataguy16

Questions 13 to 16

Window functions. The most asked topic.

13. Second highest salary

```
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER
    (ORDER BY salary DESC) AS rk
  FROM employees) t
WHERE rk = 2;
```

14. Top 2 earners per department

```
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (
    PARTITION BY dept
    ORDER BY salary DESC) AS rk
  FROM employees) t
WHERE rk <= 2;
```

15. Running total of daily revenue

```
SELECT order_date,
  SUM(amount) AS day_rev,
  SUM(SUM(amount)) OVER
    (ORDER BY order_date) AS running
FROM orders
GROUP BY order_date;
```

16. Each customer's first order

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY order_date) AS rn
  FROM orders) t
WHERE rn = 1;
```

@thedataguy16

Questions 17 and 18

Hard. These decide senior vs junior.

17. Month-on-month revenue growth %

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS mon,
         SUM(amount) AS rev
  FROM orders GROUP BY 1),
l AS (
  SELECT mon, rev,
    LAG(rev) OVER (ORDER BY mon) AS prev
  FROM m)
SELECT mon,
  ROUND(100.0 * (rev - prev) / prev, 1)
    AS growth_pct
FROM l;
```

18. Users who logged in 3 days in a row

```
WITH d AS (
  SELECT DISTINCT user_id, login_date
  FROM logins),
g AS (
  SELECT user_id,
    login_date - CAST(ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY login_date) AS INT) AS grp
  FROM d)
SELECT DISTINCT user_id
FROM g
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

Q18 trick: consecutive dates minus their row number give the same value, so each streak becomes one group.

@thedataguy16

Questions 19 and 20

Statistics in SQL and set logic.

19. Median salary per department

```
SELECT dept,
  PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY salary)
    AS median_salary
FROM employees
GROUP BY dept;
```

20. Customers who ordered in January but not February

```
SELECT customer_id FROM orders
WHERE order_date >= '2026-01-01'
  AND order_date <  '2026-02-01'
EXCEPT
SELECT customer_id FROM orders
WHERE order_date >= '2026-02-01'
  AND order_date <  '2026-03-01';
```

@thedataguy16

Your score

Count the ones you solved without looking.

- **17-20**

  Apply now. You are ahead of most candidates.

- **12-16**

  Apply, and practise window functions (13-18) daily while you interview.

- **8-11**

  Spend 2 weeks on joins and window functions first.

- **Below 8**

  Work through the basics again. Re-take this test in 3 weeks.

In the interview: say your approach before writing. "I will rank salaries within each department, then filter to rank 2 or less."

@thedataguy16